DUOC UC · ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

DSY1104 - Desarrollo FullStack II · Sección DSY1104-006D · 2026-2

Documento de Cobertura del Testing — Evaluación Parcial 2 (EP2)

Caso: Forma B – Tienda Online Sonido Vivo (Viña del Mar)

1. Identificación del Proyecto

Sección	DSY1104-006D
Grupo	Grupo 02 (Forma B)
Integrantes	1. Maximiliano Dinamarca 2. Patricio Muñoz
Contexto / Caso	Tienda Sonido Vivo (Viña del Mar) — Venta online de instrumentos musicales, equipos de sonido, control de stock y retiro en tienda.
Repositorio GitHub	`https://github.com/kottaJS/tienda-sonido-vivo`
Versión evaluada	Versión 1.0.0 (Cierre Evaluación Parcial 2)

2. Configuración del Entorno de Pruebas

Para la realización de las pruebas unitarias del frontend en React + Vite, se configuró el entorno autorizado por la asignatura en reemplazo de Jasmine/Karma:

- **Framework de pruebas:** Vitest (v2.1.9)
- **Biblioteca de componentes:** React Testing Library (`@testing-library/react` v16.0.0)
- **Entorno DOM simulado:** `jsdom` (v24.1.1) junto con `@testing-library/jest-dom` (v6.4.8)
- **Comando de ejecución de pruebas:** `npm test` (ejecuta Vitest en modo run)
- **Comando de reporte de cobertura:** `npm run test:coverage` (proveedor v8 integrado)

3. Registro de Pruebas Unitarias

N°	Archivo / Componente	Comportamiento verificado	Tipo de prueba	Resultado
1	`formatters.js`	Formatea valores numéricos a formato de moneda chilena con signo peso y puntos separadores de miles ($129.990).	Lógica / Formato	**OK**
2	`formatters.js`	Valida correctamente RUTs chilenos válidos con dígito numérico y con letra K aplicando el algoritmo Módulo 11.	Lógica / Validación	**OK**
3	`formatters.js`	Rechaza RUTs con dígito verificador adulterado, cadenas vacías o valores nulos.	Lógica / Validación	**OK**
4	`formatters.js`	Valida formato de correo electrónico mediante expresión regular, rechazando correos sin arroba o sin dominio.	Validación de datos	**OK**
5	`formatters.js`	Calcula el subtotal acumulado, total de artículos y costo de despacho ($0 para retiro en Viña y $4.990 a domicilio).	Regla de negocio	**OK**
6	`ProductoCard.jsx`	Renderiza correctamente nombre, marca, modelo, precio formateado y badge de stock recibidos mediante props.	Renderizado con props	**OK**
7	`ProductoCard.jsx`	Muestra el texto 'Agotado' y deshabilita el botón de compra cuando la propiedad stock es igual a 0.	Renderizado condicional	**OK**
8	`ProductoCard.jsx`	Ejecuta la función espía (mock onAgregar) al hacer clic en el botón de compra pasando los datos del producto.	Interacción / Mock	**OK**
9	`FiltroCategorias.jsx`	Renderiza los botones de categorías y dispara la función mock con la categoría seleccionada al hacer clic.	Interacción / Eventos	**OK**
10	`Footer.jsx`	Renderiza la información física del local (Calle Valparaíso #580, Viña del Mar), datos de contacto y enlaces.	Renderizado estático	**OK**

4. Mocks y Spies Utilizados

Se utilizaron funciones mock de Vitest (`vi.fn()`) para aislar los componentes bajo prueba de los contextos globales y verificar eventos del usuario:

- **Prueba N° 8 (`ProductoCard`):** Se simuló la función prop `onAgregar` mediante `const mockOnAgregar = vi.fn()`. Al simular el clic con `fireEvent.click`, se comprobó que fue llamada exactamente una vez (`toHaveBeenCalledTimes(1)`) y con el objeto del producto como parámetro (`toHaveBeenCalledWith(productoMock)`). Esto permite probar la tarjeta sin montar el `CartContext` completo.

- **Prueba N° 9 (`FiltroCategorias`):** Se utilizó un mock `const mockSeleccionar = vi.fn()` para comprobar que al presionar el botón de la categoría 'Bajos Eléctricos', el componente notifica al padre el valor seleccionado.

5. Resultado de Cobertura

Métrica	Resultado	Interpretación
Statements (Sentencias)	**95.83%**	Se ejecutan casi todas las instrucciones de los módulos utilitarios y componentes testeados.
Branches (Ramas)	**96.29%**	Se prueban las bifurcaciones lógicas principales (stock mayor a cero vs. stock en cero, retiro vs. despacho).
Functions (Funciones)	**100.00%**	Todas las funciones declaradas en los módulos bajo prueba son invocadas durante la suite.
Lines (Líneas)	**95.83%**	Alta cobertura de líneas efectivas de código en lógica de validación y formateo.

6. Análisis Técnico

- **¿Qué componentes o funciones quedaron mejor cubiertos?**
  El archivo `src/utils/formatters.js` obtuvo más del 95% en todas las métricas, asegurando la fiabilidad de las funciones matemáticas (Módulo 11 de RUT chileno, formateo a CLP y cálculo de despacho). A nivel visual, los componentes `ProductoCard`, `FiltroCategorias` y `Footer` alcanzaron un 98% a 100% de cobertura de líneas.
- **¿Qué funcionalidades presentan menor cobertura y por qué?**
  Las vistas completas de página (`AdminPage`, `CheckoutPage`) y el mapa interactivo de Leaflet presentan menor cobertura automatizada porque su validación se concentró en la lógica interna modular y en pruebas de integración manuales durante la navegación.
- **¿Qué error o comportamiento inesperado fue detectado gracias a una prueba?**
  Al escribir la prueba N° 2 se utilizó inicialmente un RUT de prueba con dígito 2 (`19.876.543-2`). La prueba falló de inmediato porque el dígito verificador correcto por Módulo 11 era 0. Esto permitió confirmar que la función matemática detecta con total exactitud cualquier error de digitación antes del paso a producción.
- **¿Qué prueba adicional sería conveniente implementar en una siguiente versión?**
  Sería recomendable agregar una prueba de integración para el formulario completo de Checkout, simulando la secuencia completa de ingreso de datos válidos, cálculo de despacho y despacho del evento de confirmación de pedido hacia el inventario.

7. Conclusiones

La implementación de pruebas unitarias con Vitest y React Testing Library permitió validar de forma automatizada tanto las reglas de negocio críticas (validación de identidad nacional con RUT y cálculos de compra) como el comportamiento visual de los componentes de la interfaz. El uso de mocks facilitó verificar la interacción de los componentes de manera aislada, garantizando un código robusto, desacoplado y listo para ser defendido técnicamente.